

IMPLEMENTATION GUIDE

Key instruction to dev team:

Build deterministic logic first. AI should only explain outputs, not calculate them.

Also tell them:

Do not rebuild the whole app. Refactor incrementally. Keep the current frontend and existing flows where possible, but standardise backend outputs and engine logic.

1. Implementation Map

Layered Approach

1. **Computation Library**
 - a. Build first.
 - b. Pure functions, deterministic, traceable.
 - c. Tested independently.
2. **Domain Engines**
 - a. **Viability → Survival → Stability → Growth**
 - b. Use Computation Library.
 - c. Ensure consistent input/output contracts.
 - d. Include risk flags, recommendations, interpretations, confidence, and trace.
3. **Master Aggregator**
 - a. Combine all engine outputs.
 - b. Calculate master score, classifications, consolidated risks, recommendations, interpretation, confidence.
4. **Fragility Index**
 - a. Read Master Aggregator + engine risks + dependencies.
 - b. Compute fragility dimensions, top factors, propagation risks, ASI triggers.
 - c. Output deterministic classification.
5. **Scenario / Adaptive Scenario Intelligence**
 - a. Take base engine outputs + scenario changes.
 - b. Recompute all engines + fragility + master aggregator.
 - c. Compute deltas, rank scenarios, generate ASI triggers, produce interpretation.

6. Engine Orchestrator

- a. Single entry point.
- b. Sequence: Viability → Survival → Stability → Growth → Master → Fragility → Scenario/ASI.
- c. Returns unified structured output.

7. Standard Output Contract

- a. One JSON response: frontend, AI narratives, PDF reports.
- b. Must include scores, classifications, risks, recommendations, scenarios, narratives, confidence, trace.

8. AI Narrative Layer

- a. Converts deterministic outputs into business-friendly explanations.
- b. Template-first for MVP, optional LLM polish.
- c. Validate against output contract.

9. PDF Report Generator

- a. Uses stored output + narratives.
- b. Build sections, render HTML → PDF.
- c. Save report metadata for retrieval.

10. Persistence Layer + API

- a. Store all assessments, scores, risks, recommendations, scenarios, fragility profiles.
- b. Expose endpoints:
 - i. POST /businesses/:id/assessments/master
 - ii. POST /businesses/:id/scenarios/run
 - iii. POST /businesses/:id/reports/generate
 - iv. GET /businesses/:id/assessments/:assessmentId
 - v. GET /businesses/:id/scenarios/:scenarioRunId
 - vi. GET /businesses/:id/reports/:reportId

2. Tips for Dev Team (Codex IDE)

1. Build Incrementally

- a. Start with **Computation Library → Viability Engine → Survival Engine**.
- b. Test each engine independently before connecting to Master Aggregator.

2. Use the Contracts

- a. Every engine must read/write to **standard JSON contracts**.
- b. No engine should mutate live data directly.

3. **Traceability**
 - a. Keep trace object in all outputs.
 - b. Essential for debugging, AI narratives, PDF report generation.
4. **Validation**
 - a. Include confidence, validation notes, and warnings in all outputs.
 - b. Especially for limited data or estimates.
5. **Template-first AI narratives**
 - a. For MVP, avoid hallucinations.
 - b. Only explain deterministic output.
 - c. Use LLM for optional polish.
6. **Scenario & ASI**
 - a. Run in memory; **never overwrite live ontology**.
 - b. Persist only simulation result and deltas.
7. **API Design**
 - a. Ensure endpoints map directly to **orchestrator + persistence layer**.
 - b. Keep consistent JSON response shapes across frontend, narratives, and PDF.
8. **Testing**
 - a. Unit tests for each engine, orchestrator, scenario simulation, fragility.
 - b. Include edge cases: zero revenue, missing history, extreme risks.
9. **Naming & Versioning**
 - a. Include engineVersion, rulesetVersion, promptVersion in all persisted outputs.
 - b. Facilitates backward compatibility and debugging.
10. **Progressive Build**
 - a. Codex can generate files per engine.
 - b. Build **Computation Library** → **Engine** → **Orchestrator** → **Narrative** → **PDF** → **API** in sequence.

3. Suggested Dev Flow

1. Codex generates **Computation Library** (shared formulas).
2. Generate **Viability Engine** → unit test → persist output.
3. Generate **Survival Engine** → unit test → persist output.
4. Generate **Stability Engine** → unit test → persist output.
5. Generate **Growth Engine** → unit test → persist output.

6. Generate **Master Aggregator** → integrate all engines → unit test.
7. Generate **Fragility Engine** → unit test → integrate with Master.
8. Generate **Scenario / ASI Engine** → unit test → integrate with Orchestrator.
9. Generate **Engine Orchestrator** → integrate all engines → unit test.
10. Generate **Standard Output Contract** → validate orchestrator output.
11. Generate **AI Narrative Layer** → unit test with template-first narratives.
12. Generate **PDF Report Generator** → unit test PDF rendering.
13. Create **API endpoints** → unit test integration with orchestrator and persistence layer.